AI - BASED INDUSTRIAL QUALITY INSPECTION SYSTEM

WORKING EXPLANATION-WEB AND ANDROID APPLICATION

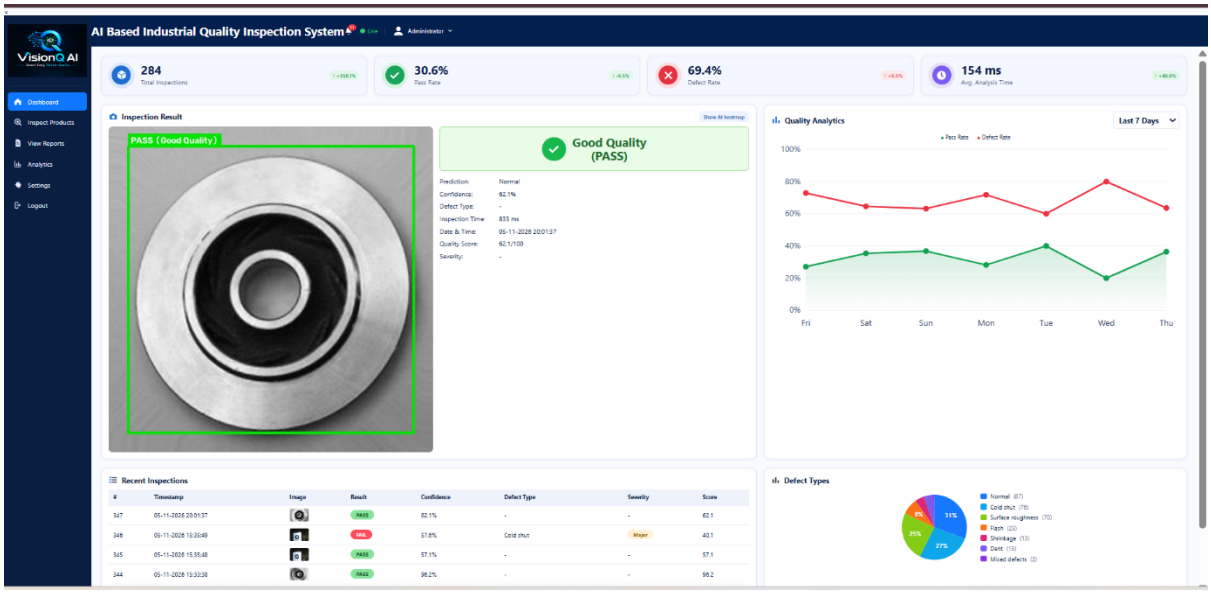
DOCUMENT PURPOSE: □A simple explanation of how the completed system is structured and how its main parts work together.

Application Types: Web application and Android mobile application.

Core Technology: Python, Flask, REST API, TensorFlow/Keras (CNN), OpenCV (Grad-CAM defect localization), Pandas, Report Lab, SQLite database, HTML/CSS/JS for web application.

1.ProjectOverview:

The AI Based Quality Inspection System (ABQIS) is designed to help a manufacturing unit automatically inspect product surface images and identify defects. Instead of depending only on manual visual checking, the system uses a Convolutional Neural Network (CNN) to classify each uploaded image as good or defective, identify the type of defect, and highlight the exact region where the defect is located.



The system recognizes defect types such as scratch, crack, porosity, pinhole, inclusion, shrinkage, surface roughness, dent, cold shut, flash and mixed defects. Every inspection result is stored in a database so that users can review history, view dashboard summaries, receive alerts and generate reports. The project is delivered as a web application that runs in a browser.

The project has two user-facing applications: A website that can be accessed through a browser and an Android application that can be accessed from a mobile device. Both applications are intended to use the same backend services and central database when configured to connect to the same server.

2. Overall System Architecture:

The system can be understood as three main layers:

1. Presentation layer: The website and Android application show screens, accept image uploads and display results.

2. Application layer: The Flask backend receives requests, validates them, calls the AI engine, applies business rules and sends responses.

3. AI layer: The TensorFlow CNN model classifies the image, and the Grad-CAM + OpenCV module produces a heat-map that shows where the defect is.

4. Data layer: The database stores users, inspection records, defect results and alert information.

Basic request flow: User → Web screen → Flask API request → Image pre-processing → CNN prediction → Grad-CAM localization → Database save → API response → Result displayed on screen.

The web page and the mobile browser use the same backend and the same database, so every device shows the same stored information.

3. How the Website Works:

A user opens the application URL in a browser and navigates through the available pages. When the user uploads an image or selects a feature, the page sends a request to the Flask backend. The backend processes the request, runs the AI model when required, reads or writes data in the database and returns the response, which the page then displays.

Typical website flow:

- Open the application URL in a browser (for example, http://127.0.0.1:5000 on the local machine).
- Enter login details on the login page, if login is enabled.
- After successful authentication, open the dashboard or a required module.
- Upload a product image for inspection.
- View the predicted defect type, confidence score and the highlighted defect area.
- Review past inspections, alerts and reports.

4. How the Android Application Works

The application uses a responsive layout and a web app manifest, so it adapts to phone and tablet screens. When opened in a mobile browser, it can be added to the home screen and launched like an installed app. The mobile screens communicate with the same Flask backend through the same API calls, and the results are shown on the mobile screen. The phone camera can be used to capture the product image directly, where this option is implemented.

Typical Android flow:

• Open the application on an Android device.

• Sign in, if authentication is enabled.

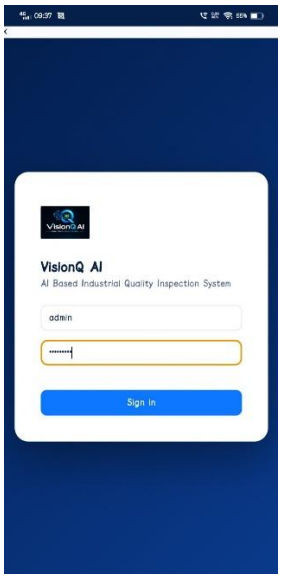
• Navigate to the available dashboard and modules.

• Request or submit information through the app.

• Receive the backend response and show the result on the mobile screen

5. Login and User Access

The login screen is the entry point for users. The application sends the entered credentials to the backend, which validates them against the user records stored in the database and returns the result. Protected pages and APIs are accessible only after a successful login, according to the authentication and role features implemented in the project. Demo users can be created using the seed script for testing and demonstration.



6. How the AI Inspection Engine Works

The AI engine is the core of the project. It works in the following steps:

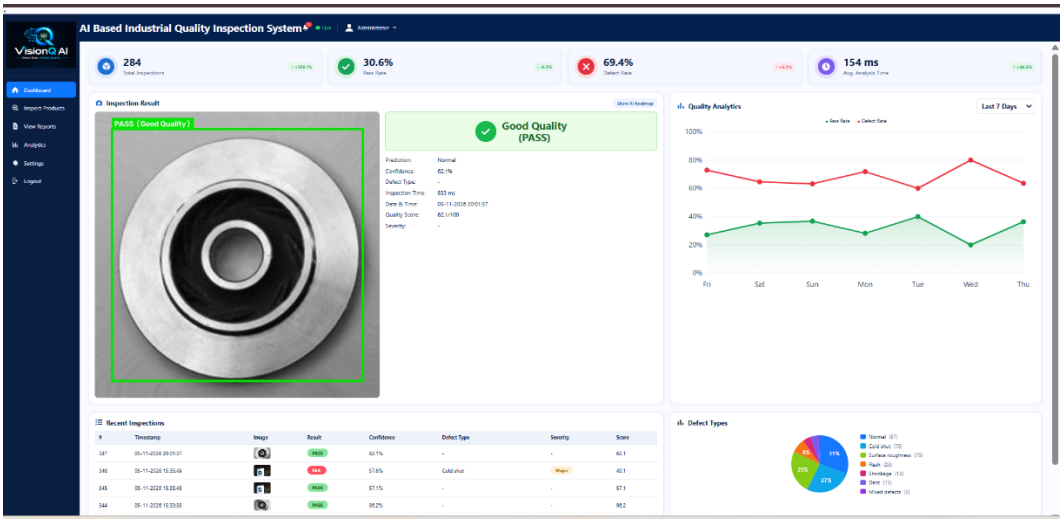
1. **Image input:** The uploaded image is read using OpenCV and resized to the model input size (128 × 128 pixels).
2. **Pre-processing:** Pixel values are scaled to the 0–1 range so that the model receives normalized data.

3. **CNN classification:** The trained CNN, built with TensorFlow/Keras, passes the image through convolution, batch normalization, ReLU activation and max-pooling blocks. Global average pooling, dropout and dense layers then produce the probability for each defect class.
4. **Defect localization:** Grad-CAM uses the last convolution layer of the model to create a heat-map showing which part of the image influenced the decision. OpenCV overlays this heat-map on the original image and marks the defect region.
5. **Result output:** The system returns the defect type, confidence score, pass/fail status and the annotated image.

7.How the Project Main Modules Works

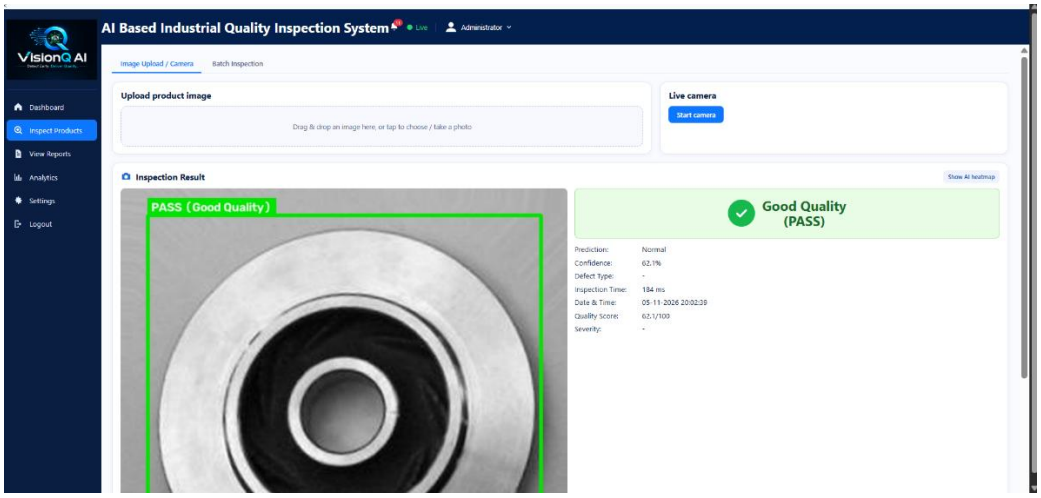
The following descriptions explain the purpose of the main modules. The exact operations available in each module depend on the screens, API endpoints and database functions implemented in the application.

Dashboard



Displays a summary of key quality information such as total inspections, number of good and defective items, defect-type distribution and recent results, with shortcuts to important sections.

Image Inspection (AI Detection)

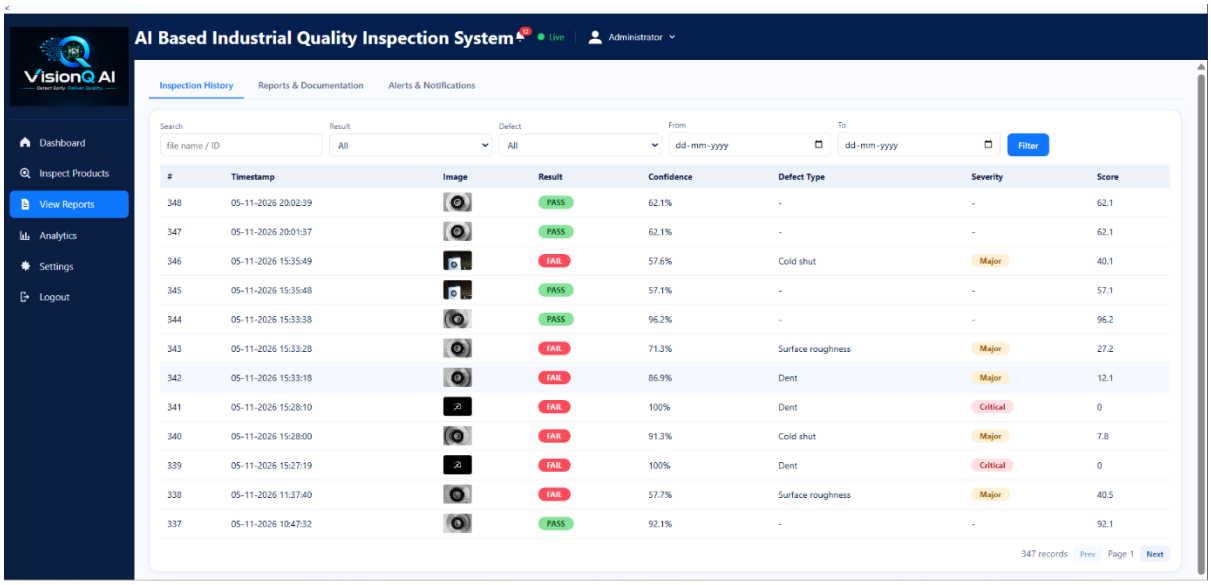


Allows the user to upload or capture a product image and run it through the CNN model. The result shows the defect type, confidence and pass/fail decision.

Defect Localization (Grad-CAM)

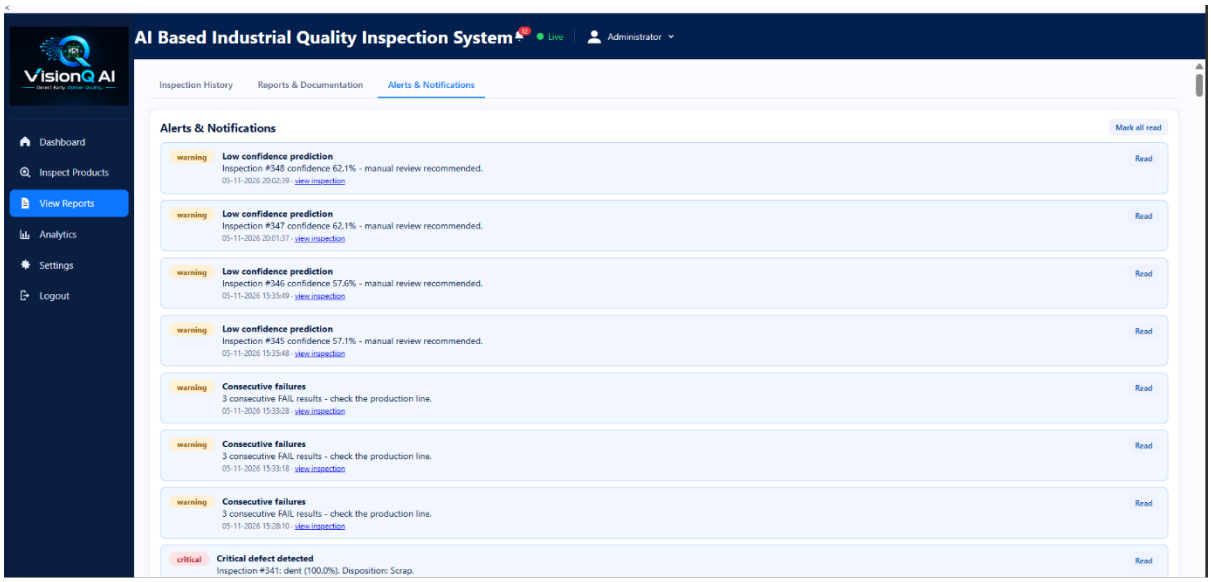
Generates a heat-map and highlighted region on the image so the user can see exactly where the defect is, which makes the AI decision easy to understand and verify.

Inspection History and Records



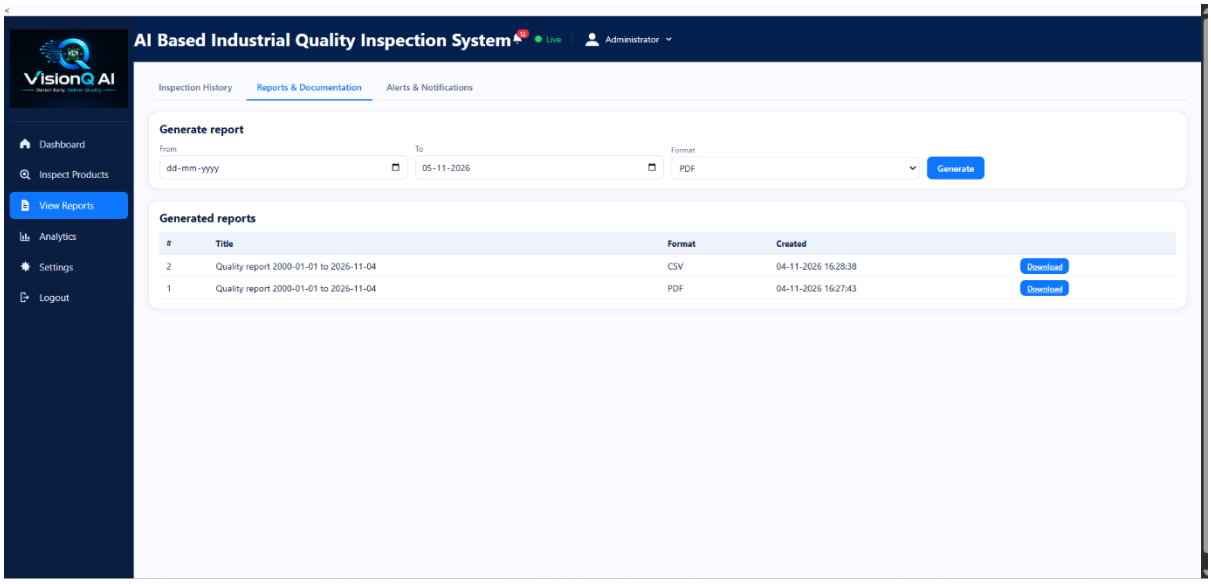
Stores every inspection with its image, result, confidence and time. Users can view, search and filter past records for traceability.

Smart Alerts and Notifications



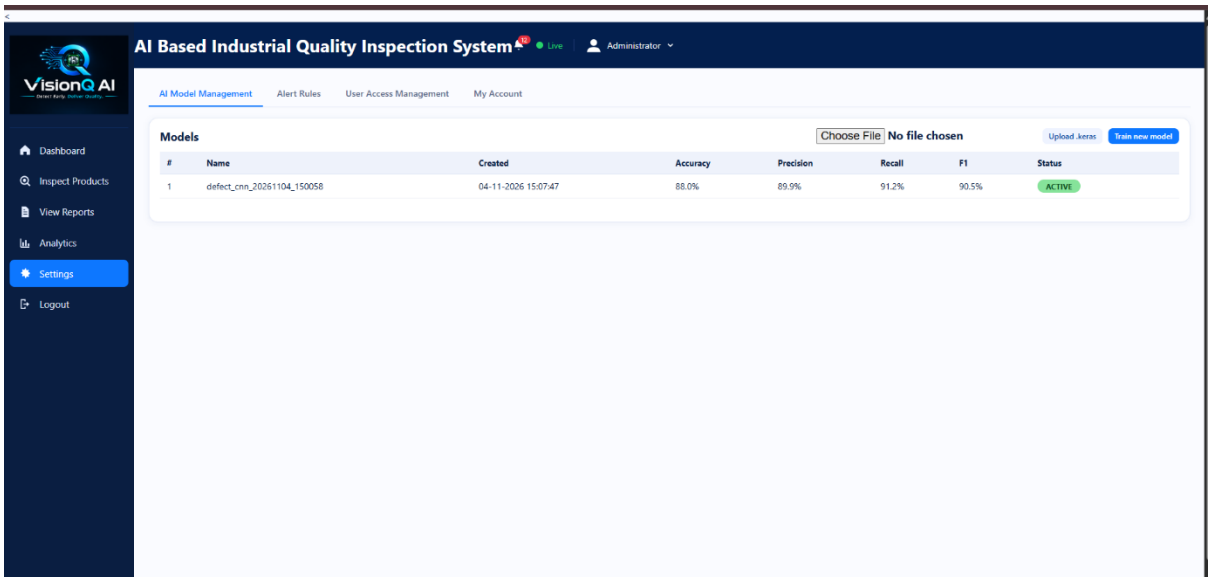
Shows warnings when configured conditions are met, such as a high defect rate, repeated occurrence of the same defect type or a critical defect being detected.

Reports and Documentation



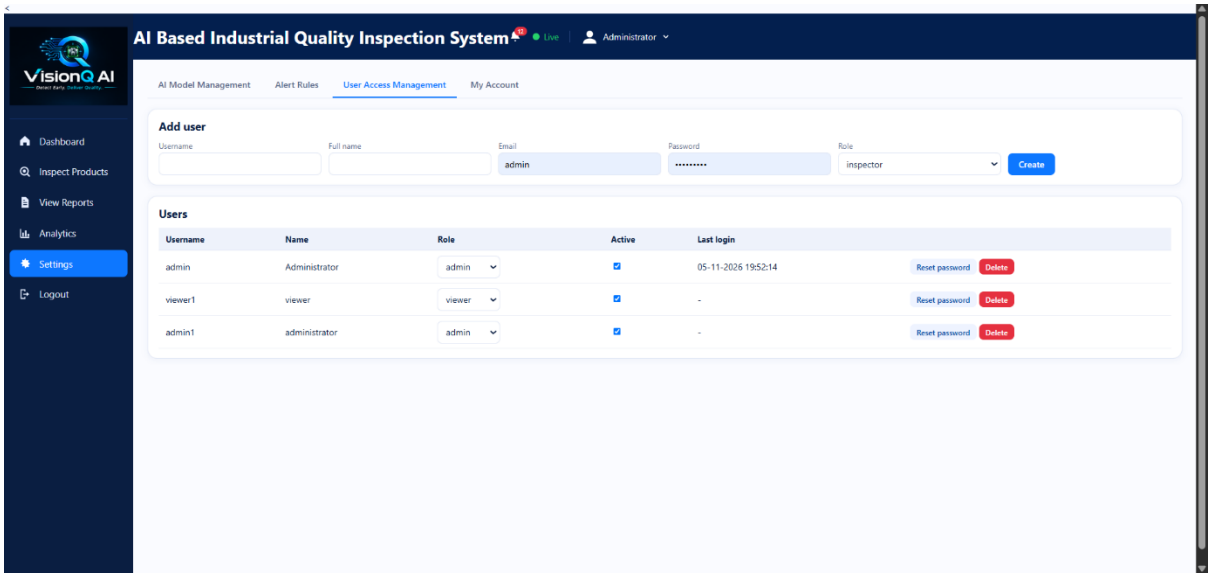
Generates quality reports from stored inspection data and exports them in supported formats such as PDF (ReportLab) or tables (Pandas), summarizing defect counts, trends and pass/fail percentages.

AI Model Management



The training script trains the CNN on this dataset and saves the trained model in the models folder, which the application loads at start-up. The model can be retrained when new defect images are added.

User and Access Management



Manages user accounts and access where these functions are implemented. Demo data and demo users can be generated with the seed script for testing.